מערכות הפעלה – בית ב'

1. הבעיה יסודית שגורמת לאי עקביות בקבצי היומן ובקובץ החשבון היא חוסר בסנכרון נכון בין הסוכנים שמעדכנים את המשאבים השותפים. בסביבה מקבילה, סוכנים מרובים לנסות לקרוא ולכתוב לקובץ החשבון בו זמנית, מה שמוביל לתנאי תחרות.

כדי לתקן זאת, נשתמש בסמפור כדי לוודא שרק סוכן יכול לגשת למשאב המשותף בכל רגע נתון. סמפור יאפשר לנו לשמור על מניעת גישה בו זמנית תוך שמירה על מקבילות.

תמונה שרץ כנדרש לאחר התיקון:

```
hodaya2002@LAPTOP-SRUSS0I8: ~
Op of A2 = -26
Op of A2 = 46
Op of A1 = 18
Op of A1 = -32
Op of A2 = 23
Op of A1 = -10
Op of A1 = -3
Op of A1 = -46
Op of A2 = -46
Op of A1 = 12
Op of A2 = 3
Op of A2 = -5
Op of A1 = 3
Op of A1 = 50
Op of A1 = -4
Op of A1 = 3
Agent A1: Successful!
Op of A2 = 10
Agent A2: Successful!


 ----- Controller -----
Sum of A1 = -285
Sum of A2 = -132

Operations O.K. Controller: 83, Account: 83
```

2. הבקר מבצע בדיקות עקביות של קבצי היומן וקובץ החשבון במרווחים קבועים או בנקודות ספציפיות בתוכנית .כדי להפוך את הבדיקות לאקראיות, נוכל במחולל מספרים אקראיים כדי לקבוע את הרווחים בבקר יבצע את בדיקותיו.

3. שימוש בפונקציה הנ"ל בסוף ה- main הוא רעיון טוב, אומנם צריך לשים לב שכל התהליכונים סיימו את עבודתם לפני שפונ' main הסתיימה.

הקוד שעונה על המשימות:

```
hodaya2002@LAPTOP-SRUSS0I8: ~
  GNU nano 6.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <time.h>
#include <errno.h>
#include <pthread.h>
#include <stdatomic.h>
#include <semaphore.h>

#include "agents.h"
/*************************************/
int init(int);
int check_point(int*);
void go_agent(char *);
int log_sum(char *);
int update(int, char *, char *);
int write_to_log(int, char *);
void fatal_error(char *, char *);
int gen_rand(int);
void *controller_thread(void *arg);
/*********************************************************************/
/*          Global Array - All Agents' names              */
char agents[MAX_AGENTS][NAMES_LEN] = AGENTS;
int p; /* number of the started agents */
pthread_t thread[MAX_AGENTS]; /* agents */
pthread_t cThread; /* control thread */
pthread_mutex_t account_mutex = PTHREAD_MUTEX_INITIALIZER; /* mutex for account file */
/*********************************************************************/

/* Reset all files, Dispatch all agents and control actions */
int main(int ac, char **av)
{
    int i = 0, num_agents = 1; /* default */
    if (ac == 2) {
        i = atoi(av[1]);  /* Check arguments */
    } else if (ac != 1) {
        fatal_error("USAGE: agents [number_of_agents]\n", NULL);
    }

    /* initiate an account file with a start sum */
    num_agents = init(i);

    /* Create control thread */
    if (pthread_create(&cThread, NULL, controller_thread, &num_agents)) {
        perror("ERROR creating control thread.");
```

```
        perror("ERROR creating control thread.");
        exit(ERROR);
    }

    /* Main process distributes work to threads (agents).
       Agents perform independent operations. Every agent updates the account file
       after each operation and logs them in a personal log file */
    for (i = 0, p = 0; i < num_agents; i++, p++) {
        if (pthread_create(&thread[p], NULL, (void *)go_agent, &agents[p])) {
            perror("ERROR creating thread.");
            --p;
        }
    }

    i = p;
    printf("Finished creating %d agent(s).\n", num_agents);
    sleep(1);

    /* The father waits for agents to return successfully */
    for (i = 0; i < p; i++) {
        pthread_join(thread[i], NULL);
    }

    /* Notify controller thread that agents are done */
    p = 0;
    pthread_join(cThread, NULL);

    /* check the consistency (log files vs. the account file) */
    check_point(&num_agents);
    return 0;
}

/* Controller thread function to perform random checks */
void *controller_thread(void *arg)
{
    int *num_agents = (int *)arg;
    while (p > 0) {
        sleep(rand() % MAX_SLEEP + 1); /* Random sleep interval */
        pthread_mutex_lock(&account_mutex);
        check_point(num_agents);
        pthread_mutex_unlock(&account_mutex);
    }
    return NULL;
}
```

```
/*  initiate the account file with the starting sum
 *  return the number of agents to be created in the range [1, MAX_AGENTS] */
int init(int num_agents)   /* Initiate auxiliary files, returns number agents */
{
    FILE *fp;
    int i;
    // srand(time(NULL)); // seeds rand()
    if (num_agents < 1 || num_agents > MAX_AGENTS) {
        printf("%d agent(s) defaults to 1\n", num_agents);
        num_agents = 1;
    }
    printf("Creating %d agent(s).\n", num_agents);
    // delete old files; exit if fails
    for (i = 0; i < num_agents; i++) {
        if ((unlink(agents[i]) < 0) && (errno != ENOENT)) {
            fatal_error("Deleting agents log", "");
        }
    }
    if ((fp = fopen(ACCOUNT, "w")) == NULL) {
        fatal_error("Creating ACCOUNT file", "");
    }
    fprintf(fp, "%d\n", START_SUM);
    fclose(fp);
    return num_agents;
}

/*********************************************************************/

int check_point(int *num_agents) /* Are agents' actions consistent? */
{
    int i, n, control_sum, sum;
    control_sum = 0;
    if ((n = rand() % MAX_SLEEP) != 0) {
        sleep(n * 3);
    }
    printf("\n\n ----- Controller ----- \n");
    if ((sum = update(0, ACCOUNT, NULL)) == ERROR) {
        return ERROR; // get the sum by updating 0
    }
    for (i = 0; i < *num_agents; ++i) {
        control_sum = control_sum + log_sum(agents[i]);
    }
    control_sum = START_SUM + control_sum;
    if (sum != control_sum) {
        fprintf(stderr, "\n Not consistent! Somebody steals?!\n");
        fprintf(stderr, "Controller: %d, Account: %d\n", control_sum, sum);
```

```c
        fprintf(stderr, "Terminating all agents!\n");
        return 1;
    }
    printf("\nOperations O.K. Controller: %d, Account: %d\n", control_sum, sum);
    return SUCCESS;
}

/*****************************************************************/
int log_sum(char *log_file)    /* Returns the sum of log_file */
{
    FILE *fp;
    int sum = 0, tmp_sum;
    if ((fp = fopen(log_file, "r+")) == NULL) {
        perror(log_file);
        printf("\nCan't Control %s agent!\n", log_file);
        return 0;
    }
    while (fscanf(fp, "%d", &tmp_sum) > 0) {
        sum = sum + tmp_sum;
    }
    fclose(fp);
    printf("Sum of %s = %d\n", log_file, sum);
    return sum;
}

/*****************************************************************/
void fatal_error(char *err_msg, char *perror_arg)  /* Prints Error messages */
{
    fprintf(stderr, "\nERROR: %s. ", err_msg);
    if (perror_arg != NULL) {
        perror(perror_arg);
    }
    exit(ERROR);
}

/*****************************************************************/

void go_agent(char *agent_name)      /* sends agent to work */
{
    int n, i, amount;
    for (i = 0; i < OPERATIONS; ++i) {
        amount = gen_rand(MAX_OPER);
        pthread_mutex_lock(&account_mutex);
        if (update(amount, ACCOUNT, agent_name) == ERROR) {
            pthread_mutex_unlock(&account_mutex);
            fatal_error("Can't Update Account File", ACCOUNT); // It includes process termination
```

```c
            fatal_error("Can't Update Account File", ACCOUNT); // It includes process termination
        } else {
            pthread_mutex_unlock(&account_mutex);
            if (write_to_log(amount, agent_name) == ERROR) { // rollback
                pthread_mutex_lock(&account_mutex);
                if (update(-amount, ACCOUNT, NULL) == ERROR) {
                    fprintf(stderr, "ERROR: Rollback failed. Agent %s\n", agent_name);
                }
                pthread_mutex_unlock(&account_mutex);
                fatal_error("Can't log operations", agent_name); // It includes process termination
            }
        }
        pthread_mutex_unlock(&account_mutex);
        printf("Op of %s = %d\n", agent_name, amount);
        if ((n = rand() % MAX_SLEEP) != 0) {
            sleep(n);
        }
    }
    p--;
    printf("Agent %s: Successful!\n", agent_name);
    return;
}

/*****************************************************************/
int gen_rand(int range) /* Generates random numbers between -range and +range */
{
    int n, sign;
    static int s = 17;
    srand(time(NULL) % s++);
    while ((n = rand()) == 0);
    sign = (n / 1111) % 2;
    n = ((n / 1117) % range) + 1;
    if (sign == 0) {
        return n;
    } else {
        return 0 - n;
    }
}

/*****************************************************************/
/* Updates account using amount. Returns new sum */
int update(int amount, char *account, char *name)
{
    FILE *fp;
    int n;
    char buf[BUF_LENGTH];
```

```c
    memset(buf, '\0', BUF_LENGTH);
    if (!(fp = fopen(account, "r"))) {
        return ERROR;
    }
    fscanf(fp, "%s", buf);
    fclose(fp);
    // if (amount != 0) printf("%s", "."); // uncomment to see the progress, not needed
    if (!(fp = fopen(account, "w"))) {
        return ERROR;
    }
    fprintf(fp, "%d", n = atoi(buf) + amount);
    fclose(fp);
    return n + amount;               /* New sum in account */
}

/*****************************************************************/
int write_to_log(int amount, char *log_file)  /* Appends amount to log_file */
{
    FILE *fp;
    if ((fp = fopen(log_file, "a")) == NULL) {
        return ERROR;
    }
    fprintf(fp, "%d\n", amount);
    fclose(fp);
    return SUCCESS;
}
```